Chapter III – Machine Learning Project

Instituto Superior de Engenharia
Politécnico de Coimbra

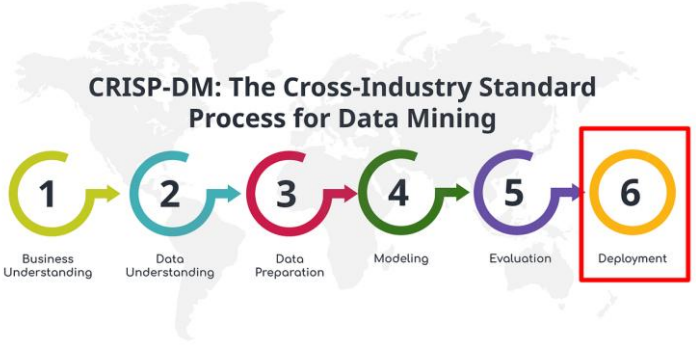
Simão Paredes
sparedes@isec.pt

Outline

- Main Phases of a Machine Learning Project:
  - Frame the problem
  - Obtain the data
    - Discover and visualize Data
  - Prepare Data for ML algorithms
  - Select and Train a Model
    - Fine tune the model
  - Deployment and Monitor

Main Phases

- CRoss Industry Standard Process for Data Mining (CRISP-DM)



https://www.linkedin.com/pulse/exploring-crisp-dm-significance-phases-considerations-data-driven-gyioc

Main Phases

1- Frame the problem

2 – Obtain the data

3- Prepare Data for ML algorithms

M1

4- Select and train a model

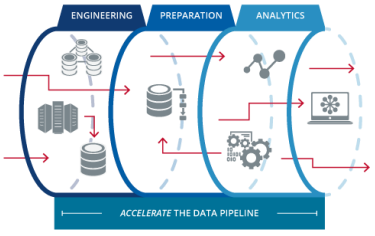
M2

5- Deployment and Monitor

Frame the Problem

Machine Learning Project

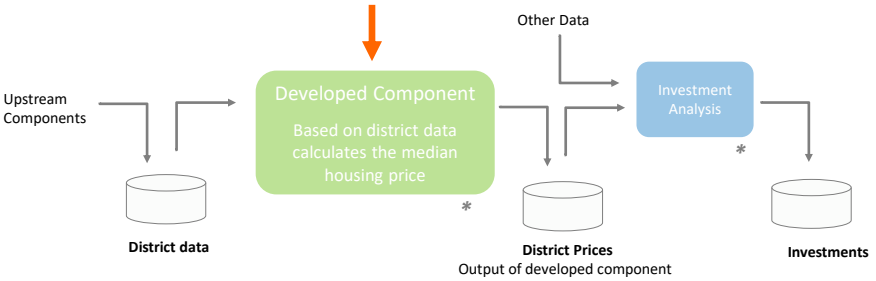
- The definition of the initial goal is critical to:
  - Frame the problem
  - Select the most proper algorithms
  - Select the performance measures , …
- A model is part of a decision pipeline
  - i.e. a sequence of data processing components in order to support decision



https://www.heavy.ai/technical-glossary/data-pipeline

Machine Learning Project

- *California Housing Prices?*
  - Estimate the median price of a house in a district

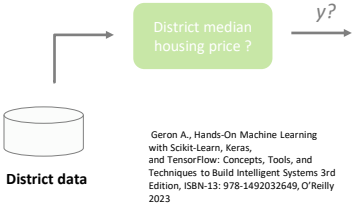


* Typically in a pipeline components run asynchronously

Output of developed component Might be an input for other algorithm that supports a decision process

Frame the Problem

- The definition of the main objetive of the system is critical as it will determine:



Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 3rd Edition, ISBN-13: 978-1492032649, O'Reilly 2023

1
Is it a supervised or unsupervised learning problem?

Supervised Learning

Labeled examples as each instance comes with the expected output, i.e. district's median housing price

Frame the Problem

2
Is it a classification or a regression task?

Regression (predict a numeric value)

Regression (multiple features (e.g. population, median income, …)) to make a prediction.

Univariate regression since only a single value (housing price) is predicted. If multiple values per district were predicted, it would be a multivariate regression.

3
Batch learning or online learning technique?

Batch learning

There is no continuous flow/input to the system.

Frame the Problem

- Framing the problem also includes the selection of **performance metrics** adapted to the nature of the problem:
  - e.g. Regression Problem
    - Root Mean Square Error:

$$\mathrm{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(h(\mathbf{x}^{(i)}) - y^{(i)})^2}$$

      - Gives how much error the system make in its predictions $\hat{y}^{(i)} = h\left(\mathbf{x}^{(i)}\right)$, where N is the number of instances, $\mathbf{X} \in R^{N,M}$, M is the number of features, h is the prediction function
      - RMSE has **the same units** as the target y
    - Mean Square Error:

$$\mathrm{MSE}(\mathbf{X}, h) = \frac{1}{N}\sum_{i=1}^{N}(h\left(\mathbf{x}^{(i)}\right) - y^{(i)})^2$$

Frame the Problem

- Alternatively, a measure less prone to outliers (once in RMSE the errors are squared before they are averaged) might be applied
  - Mean Absolute Error:

$$\mathrm{MAE}(\mathbf{X}, h) = \frac{1}{N}\sum_{i=1}^{N}\left|h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right|$$

- These metrics are valid alternative to measure the distance between two vectors $\hat{\mathbf{y}}$ (predictions) and $\mathbf{y}$ (target values)
  - These are possible **cost functions** that need **to be minimized**
    - **Learning process !!**

Obtain The Data

Obtain the Data

- The process to obtain the required data can be very troublesome
  - Data spread across multiple tables/documents/files
  - Unauthorized access to the data (e.g. General Data Protection Regulation (GDPR))
  - ...

- Data download (e.g. 'housing.csv')

Label

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY
...	...	...	...	...	...	...	...	...	...	...

Obtain the Data

- Each instance (row) represents a district
  - A global view of data must be obtained:
    - Total number of rows
    - Features / Attributes
    - Each Attribute Type
    - Missing values
    - ...

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

  - Some Findings:
    - All features (except 'ocean_proximity') are numerical
    - 'total_bedrooms' contains some missing data

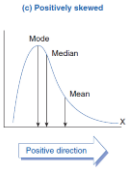
Obtain the Data

- Depicting 'ocean_proximity':
  - It is a categorical attribute
  - Different values of '*ocean_proximity*'
    - 5 unique values
    - number of instances per value

```
1 df['ocean_proximity'].value_counts()
```

```
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
Name: ocean_proximity, dtype: int64
```

- Summary of numerical attributes:

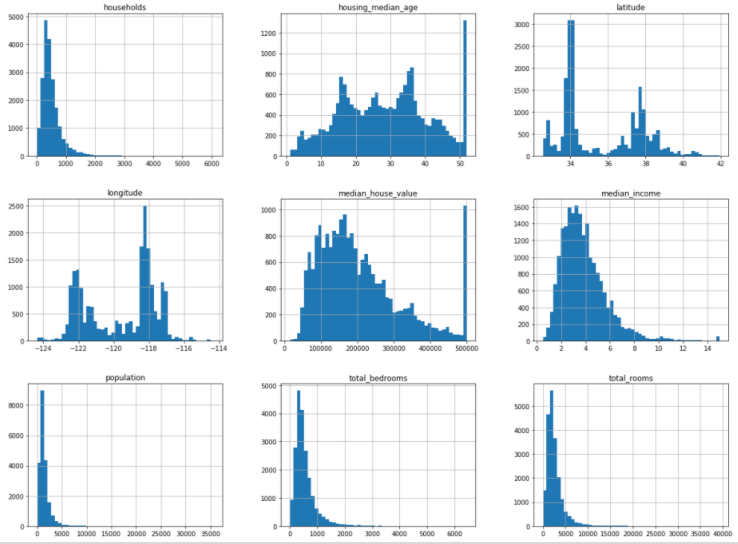


e.g. 25% of the districts have a housing median age < 18

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	1425.476744	499.539680	3.870671	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	1132.462122	382.329753	1.899822	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	787.000000	280.000000	2.563400	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.000000	409.000000	3.534800	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	1725.000000	605.000000	4.743250	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.000000	6082.000000	15.000100	500001.000000

Obtain the Data

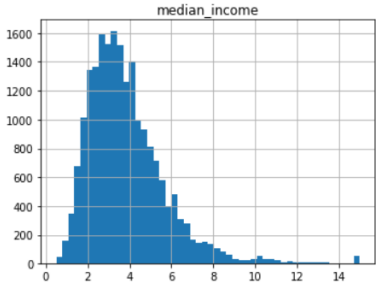
- The initial perception of the dataset **can be reinforced** with the graphical representation (e.g. histograms) of each numerical attribute:



50 bins

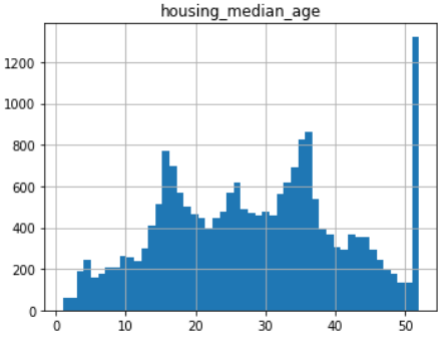
Obtain the Data

- What kind of information can be extracted from histograms?
  - **median_income**
    - It is presented in tens of thousands of USD, the lower incomes were limited at 0.5 and the higher incomes were caped at 15 ($150000).
    - This means that data was preprocessed, which is very common in real data.
      - The preprocess procedure must be **always known**.



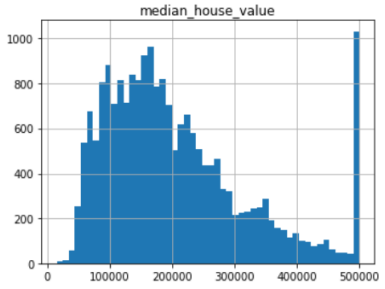
Obtain the Data

- What kind of information can be extracted from histograms?
  - **housing_median_age**
    - It was caped at 50 years old.
    - The impact of this procedure must be carefully evaluated.



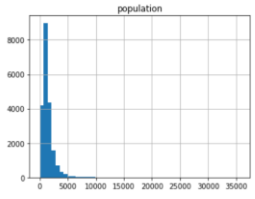
Obtain the Data

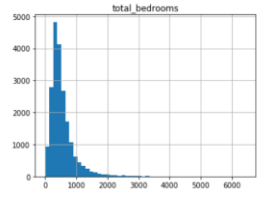
- What kind of information can be extracted from histograms?
  - **median_house_value**
    - It was limited to $500000
      - In this particular situation this artificial limitation might be a serious problem as this is the **target attribute (label)**
    - The ML algorithm may learn prices only up to that limit.
      - Solutions?
        - Collect the proper labels
        - Exclude those instances

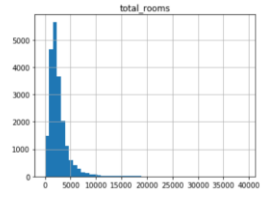


Obtain the Data

- What kind of information can be extracted from histograms?
  - The several attributes **have very different scales**, which depending on the applied algorithm **can originate some prediction problems**.
  - Some histograms 'population'; 'total_bedrooms'; 'total_rooms' are **highly skewed** (extend much further to right of median than to the left)
    - To some machine learning algorithms this can be an additional difficulty







  - Scaling?
  - Normalization?
  - Box-Cox transformation, other … ??

Obtain the Data

- Depending of the dimension of the dataset and of the validation strategy that is going to be implemented, the test set should be created in this phase.
  - the main goal is to **set some part of data aside** to avoid any previous pattern detection/analysis in this data.



https://www.oreilly.com/library/view/machine-learning-algorithms/9781789347999/f394f9d6-19dc-4103-8715-8895ef4d10b8.xhtml

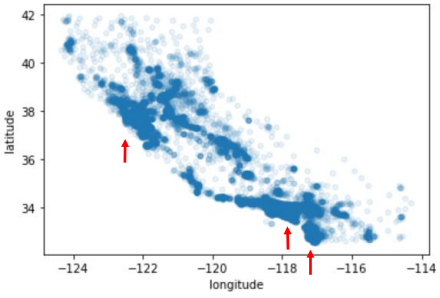
  - The test set must be representative of all the data.
    - Random Selection
    - **Global sampling vs. Stratified sampling:**
      - population is **randomly** divided into homogeneous subgroups (strata) and the proper number of instances are randomly sampled from each stratum. (e.g.: male, female)

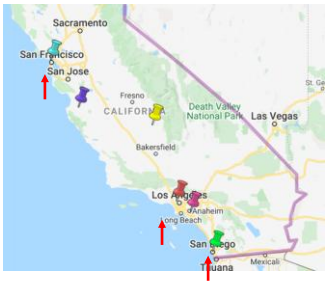
Obtain the Data

Visualize Data (additional visualizations)

Visualize Data

- Visualization of data is very useful/important to get some more insights about the distribution of data
  - Example:
    - Geographical location of instances: plot latitude/longitude

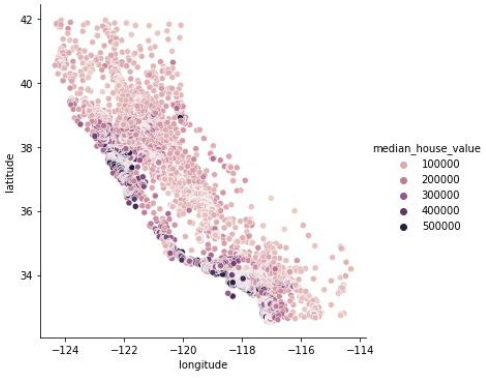




It is possible to identify the high-density areas

Visualize Data

- In order to investigate some pattern of **median_house_value** might be very useful to have a first glance of data distribution



Some findings:

- The housing prices **are related to the location** (close to ocean)
- The housing prices are also **related with the population density**
- The ocean proximity might be useful but **cannot be applied as a rule** (as confirmed in the north)

Visualize Data

- It is also very useful to analyze correlations between each attributes and the target (label)
  - Matrix of correlation provides the correlation between all pair of attributes
    - Remember!
      - **High Correlation** between:
        - a feature/attribute and the label is **desired!**
        - two features is **undesired!**

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
longitude	1.000000	-0.924664	-0.108197	0.044568	0.069608	0.099773	0.055310	-0.015176	-0.045967
latitude	-0.924664	1.000000	0.011173	-0.036100	-0.066983	-0.108785	-0.071035	-0.079809	-0.144160
housing_median_age	-0.108197	0.011173	1.000000	-0.361262	-0.320451	-0.296244	-0.302916	-0.119034	0.105623
total_rooms	0.044568	-0.036100	-0.361262	1.000000	0.930380	0.857126	0.918484	0.198050	0.134153
total_bedrooms	0.069608	-0.066983	-0.320451	0.930380	1.000000	0.877747	0.979728	-0.007723	0.049686
population	0.099773	-0.108785	-0.296244	0.857126	0.877747	1.000000	0.907222	0.004834	-0.024650
households	0.055310	-0.071035	-0.302916	0.918484	0.979728	0.907222	1.000000	0.013033	0.065843
median_income	-0.015176	-0.079809	-0.119034	0.198050	-0.007723	0.004834	0.013033	1.000000	0.688075
median_house_value	-0.045967	-0.144160	0.105623	0.134153	0.049686	-0.024650	0.065843	0.688075	1.000000

Visualize Data

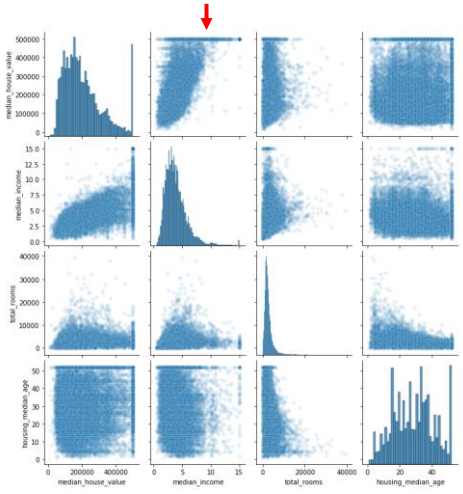
- Analyzing the correlations of **all the features** with the **label**:

```
median_house_value    1.000000
median_income         0.688075
total_rooms           0.134153
housing_median_age    0.105623
households            0.065843
total_bedrooms        0.049686
population           -0.024650
longitude            -0.045967
latitude             -0.144160
Name: median_house_value, dtype: float64
```

  - Some findings
    - The **median_income** presents a high correlation with me **median_house_value**
    - All the remaining variables present low correlation values.
  - The analysis of the most correlated variables should be further explored in order to get a deeper knowledge of the dataset

Visualize Data

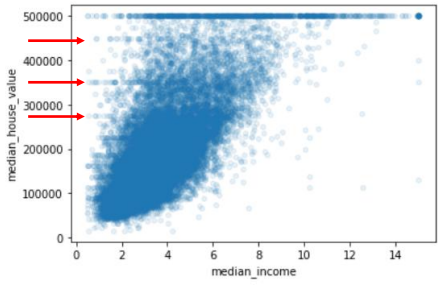
- It is possible to visualize the relationships between the most correlated variables



  - Some findings:
    - It is clear that there is a strong correlation between the **median_house_value (label)** and **media_income (feature)**

Visualize Data

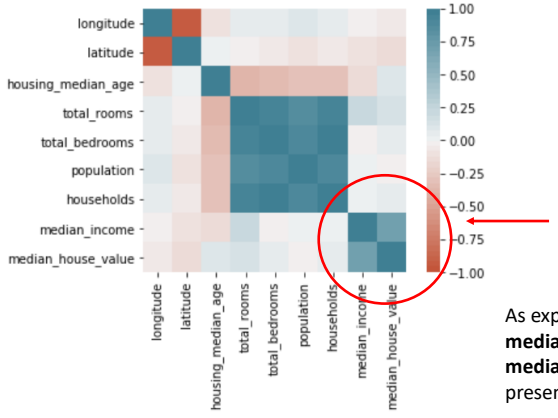
- The most correlated attribute:
  - **media_income/ median_house_value**



  - Some findings:
    - The growing trend is clear (strong correlation)
    - The limit of $500000 is also very clear.
    - However, other horizontal lines that may indicate other artificial limits are also visible. This must be clarified in order to prevent that algorithms are fed with wrong data.

Visualize Data

- Correlation can be presented through an heatmap (seaborn)
  - Usually very useful, the heatmap facilitates the identification of correlation levels.



As expected, **median_income** and **median_house_value** present high correlation

Obtain the Data

Visualize data
Derivation of New Features

Derivation of New Features

- New features created from one or more available variables
  - The main goal is to create **new features** that might be more useful to the prediction process.
  - These new features are always created based on current data
    - Example:
      - The **total number of rooms** in a district is not very useful, probably the **number of rooms per house** would be more valuable for the prediction process

```
1 df['rooms_house']=df['total_rooms']/df['households']
```


Derivation of New Features

- New features created from one or more available variables
  - rooms_house
  - The same situation with the **total number of bedrooms**:
    - Probably a feature that provides a **ratio between the total of bedrooms and the total number of rooms** would be more significant
  - A new feature could relate population with households:
    - For instance, **people per household** also seems more interesting.

Derivation of New Features

- Preliminary Evaluation of New features
  - Correlation analysis

```
median_house_value          1.000000
median_income               0.688075
rooms_per_household         0.151948
total_rooms                 0.134153
housing_median_age          0.105623
households                  0.065843
total_bedrooms              0.049686
population_per_household   -0.023737
population                 -0.024650
longitude                  -0.045967
latitude                   -0.144160
bedrooms_per_room          -0.255880
Name: median_house_value, dtype: float64
```

Some findings:

- The **bedrooms_per_room** is more correlated than the raw attributes.
  - It seems that houses with a lower bedroom/room ratio tend to be more expensive.
- The number of **rooms per house** is also more correlated than the **total_rooms**
- **population_per_household** is poorly correlated with **median_house_value**

  - Iterative process!!

Prepare Data for ML Algorithms

Training/Test Set

Training set / Test Set

- Splitting the dataset creating a training set and a test set:



Dataframe

```
1 df.shape
```

```
(20640, 10)
```

Original dataframe
'housing.csv'

=>

80%

```
1 train_set.shape
```

```
(16512, 10)
```

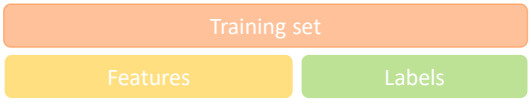
```
1 test_set.shape
```

```
(4128, 10)
```

20%

Training set / Test Set

- Considering **only the training set**:
  - A set of instances (only with attributes / features)
  - The respective labels




```
1 train_labels=train_set[['median_house_value']].copy()
```

```
1 train_labels.info()
```

Labels

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 16512 entries, 15961 to 235
Data columns (total 1 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   median_house_value  16512 non-null  float64
dtypes: float64(1)
memory usage: 258.0 KB
```

Training set / Test Set

```
1 train_instances=train_set.drop('median_house_value',axis=1).copy()
```

```
1 train_instances.info()
```

Features

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 16512 entries, 15961 to 235
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           16512 non-null  float64
 1   latitude            16512 non-null  float64
 2   housing_median_age  16512 non-null  float64
 3   total_rooms         16512 non-null  float64
 4   total_bedrooms      16349 non-null  float64
 5   population          16512 non-null  float64
 6   households          16512 non-null  float64
 7   median_income       16512 non-null  float64
 8   ocean_proximity     16512 non-null  object
dtypes: float64(8), object(1)
memory usage: 1.3+ MB
```


Prepare Data for ML Algorithms

Training/Test Set
Missing Data

Missing Data

- Missing values
  - Tree different Options
    - Delete instances
    - Delete the attribute
    - Imputation of a value
      - e.g. Median - **ONLY** applied to numeric features



```
1 train_instances=train_set.drop('median_house_value',axis=1).copy()
```

```
1 train_instances.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 16512 entries, 15961 to 235
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           16512 non-null  float64
 1   latitude            16512 non-null  float64
 2   housing_median_age  16512 non-null  float64
 3   total_rooms         16512 non-null  float64
 4   total_bedrooms      16349 non-null  float64
 5   population          16512 non-null  float64
 6   households          16512 non-null  float64
 7   median_income       16512 non-null  float64
 8   ocean_proximity     16512 non-null  object
dtypes: float64(8), object(1)
memory usage: 1.3+ MB
```

Missing data: Imputation

Drop non numeric features

Missing Data

- Data frame **after imputation process**
  - Numeric features https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html

```
1 train_num.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 16512 entries, 15961 to 235
Data columns (total 8 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           16512 non-null  float64
 1   latitude            16512 non-null  float64
 2   housing_median_age  16512 non-null  float64
 3   total_rooms         16512 non-null  float64
 4   total_bedrooms      16512 non-null  float64
 5   population          16512 non-null  float64
 6   households          16512 non-null  float64
 7   median_income       16512 non-null  float64
dtypes: float64(8)
memory usage: 1.1 MB
```

After Imputation: No missing data

  - The imputation process should be applied to the entire dataset (training + test)

Prepare Data for ML Algorithms

Training/Test Set
Missing Data
Encoding

Encoding

- The different categories of categorical attributes have to be **encoded** into numbers (int)
  - e.g. 'ocean_proximity'
    - Number of different categories

```
1 train_instances['ocean_proximity'].value_counts()
```

```
<1H OCEAN     7341
INLAND        5193
NEAR OCEAN    2136
NEAR BAY      1837
ISLAND           5
Name: ocean_proximity, dtype: int64
```

```
1 train_instances_cat
```

	ocean_proximity
15961	NEAR BAY
1771	NEAR BAY
16414	INLAND
5056	<1H OCEAN
8589	<1H OCEAN
...	...
10955	<1H OCEAN
17289	<1H OCEAN
5192	<1H OCEAN
12172	INLAND
235	NEAR BAY

16512 rows × 1 columns

→ Transform these values into numeric

Encoding operation!

Encoding

- The different categories of categorical attributes must be **encoded** into numbers
  - Ordinal Encoding: *from sklearn.preprocessing import **OrdinalEncoder***

```
1 train_instances_cat
```

	ocean_proximity
15961	NEAR BAY
1771	NEAR BAY
16414	INLAND
5056	<1H OCEAN
8589	<1H OCEAN
...	...
10955	<1H OCEAN
17289	<1H OCEAN
5192	<1H OCEAN
12172	INLAND
235	NEAR BAY

16512 rows × 1 columns

Original Values

```
1 train_cat
```

	ocean_proximity
15961	3.0
1771	3.0
16414	1.0
5056	0.0
8589	0.0
...	...
10955	0.0
17289	0.0
5192	0.0
12172	1.0
235	3.0

16512 rows × 1 columns

Ordinal Encoded Values

Problem?

Erroneous distances!

Encoding

- Actually:

The ordinal encoding might create a problem as ML algorithms may assume that two nearby values are more similar than two distant values

  - One-hot encoding: *from sklearn.preprocessing import **OneHotEncoder***
    - Create a one binary attribute per category:
      - One attribute is equal to 1 and the remaining are 0

Original Values

```
1 train_instances_cat
```

	ocean_proximity
15961	NEAR BAY
1771	NEAR BAY
16414	INLAND
5056	<1H OCEAN
8589	<1H OCEAN
...	...
10955	<1H OCEAN
17289	<1H OCEAN
5192	<1H OCEAN
12172	INLAND
235	NEAR BAY

16512 rows × 1 columns

```
1 train_cat
```

```
array([[0., 0., 0., 1., 0.],
       [0., 0., 0., 1., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

One Hot Encoded Values

Array derived from a sparse matrix

Encoding

- To handle categorical attributes to hundred or thousands of categories the output is a SciPy **Sparse Matrix** (only stores the location of nonzero elements)
  - It is possible to convert the sparse matrix to a NumPy array
    - Output of One Hot Encoding:

```
array([[0., 0., 0., 1., 0.],
       [0., 0., 0., 1., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html

https://machinelearningmastery.com/one-hot-encoding-for-categorical-data/

    - Encoding should be applied to the entire dataset (training + test)

Prepare Data for ML Algorithms

Training/Test Set
Missing Data
Encoding
Scaling

Scaling

- Usually, machine learning **present some performance problems** when the input numerical attributes **have very different scales**
  - Different approaches to address this problem:
    - min-max scaling
      - Values are rescaled to interval [0,1]

$$x_i' = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

    - Standardization

$$\widehat{z_i} = \frac{x_i - \mu}{\sigma}$$

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html
https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
https://machinelearningmastery.com/standardscaler-and-minmaxscaler-transforms-in-python/

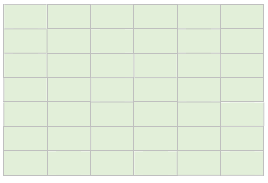
      - Scaling should also be applied to the entire dataset (training + test)

Prepare Data for ML Algorithms

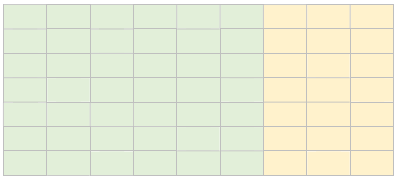
Training/Test Set
Missing Data
Encoding
Scaling
New Features

New Features

- Frequently, the creation/consideration of **new features** is very important for the learning process.
  - Custom transformer *from sklearn.base import BaseEstimator,TransformerMixin*



Original Data



Original Data + New Features

rooms per house
bedrooms per rooms
people per household

  - All the transformations need to be performed during this preprocessing phase and should be applied to the entire dataset.

New Features

- Custom Transformer
  - It allows the transformation of data, e.g the creation of **new features**
  - A Custom Transformer is an **estimator** and a **transformer**
    - A sklearn's class with the **.fit()** method is considered an estimator. The **.fit()** method allows an object to learn from the data.
    - A transformer has the **.fit()** method and the **.transform()** method that transforms the data in some way.
      - The class inherits from both classes BaseEstimator and TransformerMixin imported from sklearn.base
  - It is created to be incorporated in a transformation pipeline

```
1 import numpy as np
2 from sklearn.base import BaseEstimator,TransformerMixin
3
4 class NewAttributes(BaseEstimator,TransformerMixin):
```

    - https://scikit-learn.org/stable/modules/generated/sklearn.base.BaseEstimator.html
    - https://scikit-learn.org/stable/modules/generated/sklearn.base.TransformerMixin.html

Prepare Data for ML Algorithms

Training/Test Set
Missing Data
Encoding
Scaling
New Features
Imbalanced Datasets

Imbalanced Dataset

- Training a ML model on an imbalanced dataset creates new challenges to the learning process.
  - Most standard ML algorithms assume or expect balanced class distributions or equal misclassification costs
    - When present with severely imbalanced datasets, these algorithms present important performance limitations
      - Imbalanced ratio ranges from 1:4 up to 1:100
      - Extremely imbalanced datasets (frequent in real-life applications) may vary from 1:1000 up to 1:10000



https://towardsdatascience.com/having-an-imbalanced-dataset-here-is-how-you-can-solve-it-1640568947eb

Imbalanced Dataset

- With imbalanced datasets:
  - The **model tends to be more focused** on the normal examples (majority class) neglecting the rare events (minority classes)
    - Due to the problem context, there is a scarce representation of the most important cases/ minority classes (e.g. frauds, cancer, ….)

*Models will **be biased** towards the majority class, when usually the **minority class is the one more important**, as despite its rareness it carries important and useful knowledge.*

Applied Predictive Modeling 1st ed. 2013, Corr. 2nd printing 2018 Edition

Imbalanced Dataset

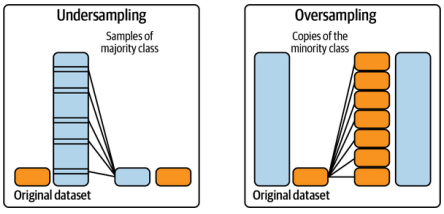
- There are several approaches to imbalanced datasets that can be organized in 3 main categories:

Data Level Methods	Algorithm-level methods	Hybrid methods
Changing the data distribution to make ML algorithms focus on the cases that are most relevant for the user.	**Changing the existing algorithms** to provide a better fit to the imbalanced training data. The cost of misclassifying a minority class is higher than the cost of classifying an instance from the majority class.	**Combine the advantages** of both Data and Algorithm level methods

Imbalanced Dataset

- Data level methods
  - Main Approach: **Resampling**
    - For classification problems, **changing the class distribution of the training data may improve** classifiers performance on an imbalanced context.
      - However, the identification of the **optimal distribution is not straightforward** as it is a domain dependent decision. In fact:
        - *for classification tasks a perfectly balanced distribution does not always provide optimal results*

(*Weiss and Provost, Journal of Artificial Intelligence Research 19 (2003) 315-354*)



https://www.oreilly.com/library/view/designing-machine-learning/9781098107956/ch04.html

Imbalanced Dataset

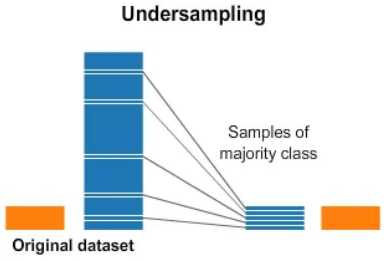
- Data level methods
  - Several types of resampling can be performed
    - The two approaches most frequently applied:

Random under/over Sampling

Synthesizing New Data

Imbalanced Dataset

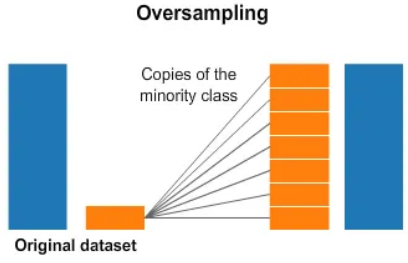
- Random **Undersampling**
  - Removal data from the majority class of **the training data** set reducing the sample size.
  - This is a very simple procedure, but might be very effective
    - However, when a random set of instances from majority class are discarded it can lead to the elimination of useful instances and consequently to a worse performance



https://www.researchgate.net/figure/Differences-between-undersampling-and-oversampling_fig1_341164819

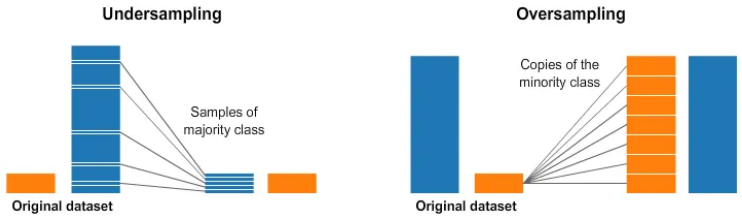
Imbalanced Dataset

- Random oversampling
  - A random set of copies of **minority class** instances is added to the **training data** increasing the sample size.
  - It is also a simple approach, but also might be effective.
    - However, this repetition of instances may increase the likelihood of **overfitting** in particular for higher over-sampling rates:
      - It may decrease the classifier performance and increase the computational effort.



Imbalanced Dataset

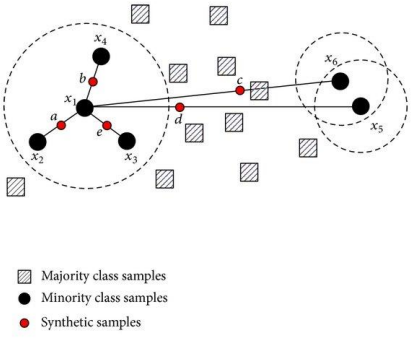
- These two techniques can be carried out though more informed methods than purely random approaches, e.g.:
  - undersampling based on a distance criteria
  - identification of possible noisy instances
  - …



  - **Resampling should be applied only to the training dataset**, keeping the test set with the original distribution

Imbalanced Dataset

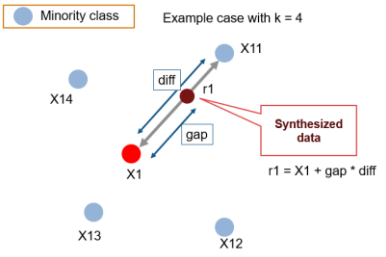
- Synthetic Minority Oversampling Technique (SMOTE)
  - creates artificial data (new instances) based on the feature space similarities between existing minority samples.
  - For the minority samples subset $S_{min} \in S$:
    - the K nearest neighbours of instance $\mathbf{x}_i \in S_{min}$ are defined as the K elements of S_{min} whose the Euclidean distance to $\mathbf{x}_i$ have the lowest value along the p dimensions of the feature space.
    - Synthetic samples are created:



https://www.researchgate.net/figure/The-schematic-of-NRSBoundary-SMOTE-algorithm_fig1_287601878

Imbalanced Dataset

- SMOTE algorithm



https://www.analyticsvidhya.com/blog/2020/10/overcoming-class-imbalance-using-smote-techniques/

- One of the K nearest neighbor ($\hat{\mathbf{x}}_i$) is randomly selected, the respective vector difference ($\hat{\mathbf{x}}_i - \mathbf{x}_i$) is multiplied by a random number δ in interval $]0,1[$ and is added to $\mathbf{x}_i$

$$\mathbf{x}_{new} = \mathbf{x}_i + (\hat{\mathbf{x}}_i - \mathbf{x}_i) \times \delta$$

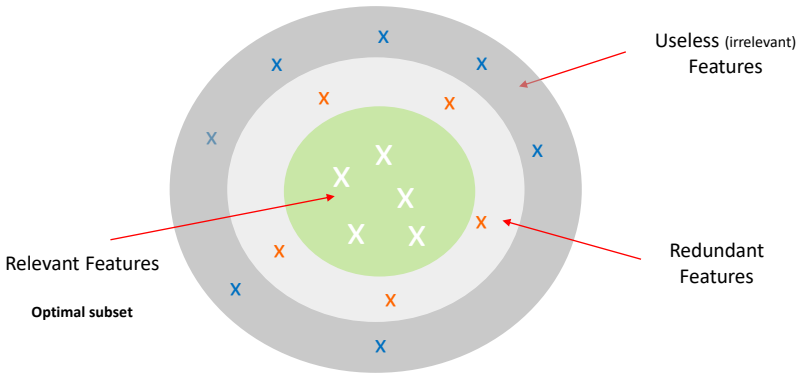
  - The resulting instance is a point along the line segment joining $\mathbf{x}_i$ and $\hat{\mathbf{x}}_i$

Prepare Data for ML Algorithms

Training/Test Set
Missing Data
Encoding
Scaling
New Features
Imbalanced Datasets
Feature Selection

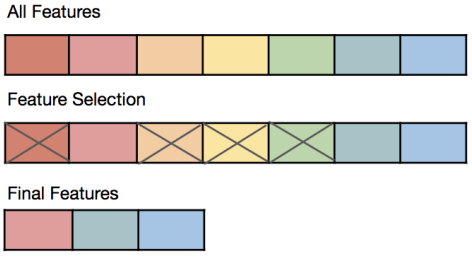
Feature Selection

- Feature selection is **the process of reducing the number of features** when developing a predictive model.
  - Usually, high dimensional data contains features that are **irrelevant** or **redundant** for the ML algorithm's performance



Feature Selection

- Curse of dimensionality — Overfitting
  - A high number of attributes favors **overfitting**, the model has a high performance in the training set but does not generalize. Useless!!
  - A high number of features increases the computational cost of modeling.



Feature Selection

- The goal is to eliminate some of the features or give them different weights.

"Let $\mathbf{x}$ be the original set of features, with cardinality $\#\mathbf{x} = n$, the continuous feature selection problem refers to the assignment of weights w_i to each feature $x_i \in \mathbf{x}$ in such a way that the order corresponding to its theoretical relevance is preserved".

L. C. Molina, L. Belanche, and À. Nebot, "Feature selection algorithms: A survey and experimental evaluation," in Data Mining, 2002. ICDM 2003. Proceedings. 2002 IEEE International Conference on, pp. 306–313, IEEE, 2002.

- By this definition, **the use of binary weights** represents the elimination of **irrelevant features**, and consequently it leads to the selection of a subset of the initial features.
  - Deciding if a feature is relevant depends on the selected evaluation measures/approaches.
- **Very important!**
  - **An optimal subset of features may not be unique** as it may be possible to achieve the same classifier performance using different sets of features.

Feature Selection

- Feature Selection
  - Selects a subset of input features from the dataset based on two main approaches:

Unsupervised	Supervised (target variable)
Does not use the target variable (e.g. remove redundant features through correlation)	**Uses the target variable** Wrapper, Filter, Intrinsic

Feature Selection

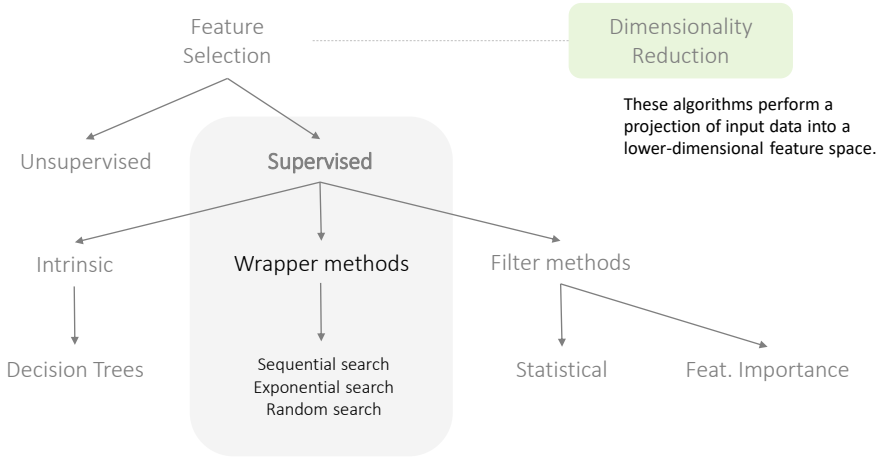
- Feature Selection

Supervised
(target variable)

  - Wrapper
    - Search for well-performing subsets of features
      - e.g. sequential search; random search
  - Filter
    - Select subsets of features based on their relationship with the target
      - e.g statistical methods; feature importance methods
  - Intrinsic:
    - Algorithms that perform automatic feature selection during training
      - e.g. Decision Trees

Feature Selection

- Overview of Feature Reduction Techniques



https://machinelearningmastery.com/feature-selection-with-real-and-categorical-data/

Feature Selection

- Wrapper methods
  - Search algorithm that tries to find the best weights for the features **using the classification algorithm itself** in order to estimate the weights' accuracy.
    - The focus of wrapper is the classifier.
    - It performs a search in the space of possible weights for features evaluating its quality (e.g. classification error) **by using the classifier as reference**.
  - Wrapper methods are usually classified into:

Sequential Selection Algorithms

Heuristic Search Algorithms

Feature Selection

- Wrapper methods

Sequential Selection Algorithms

Start with an **empty (full)** set and **add (remove)** features until the maximum of objective function is reached

Heuristic Search Algorithms

Evaluate different features subsets to **optimize the objective function**, e.g. genetic algorithms

These different subsets are generated either by searching in a specific search space or by generating solutions to the optimization problem.

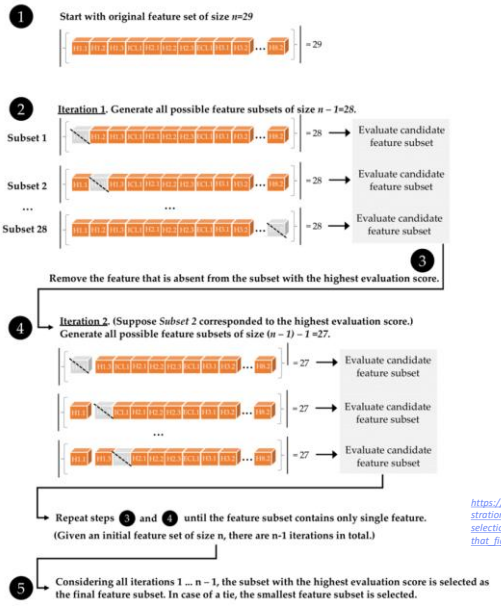
- The main drawback of wrapper methods is related with computation effort required to obtain the feature subset, as for each subset evaluation a **new model is created.**
  - The risk of overfitting should also be considered/prevented.

Feature Selection

- Wrapper methods
  - Example:

Sequential Selection Algorithms

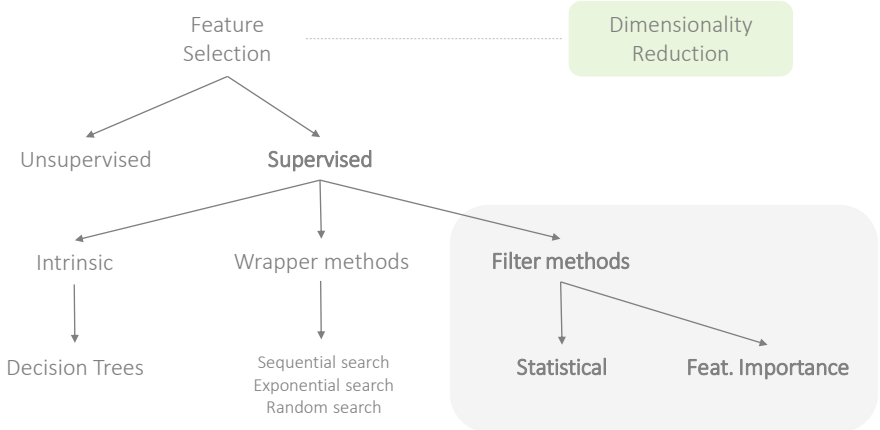
Backward Sequential Feature Selection



https://www.researchgate.net/figure/Illustration-of-backward-sequential-feature-selection-for-identifying-feature-subsets-that_fig4_339891207

Feature Selection

- Overview of Feature Reduction Techniques



https://machinelearningmastery.com/feature-selection-with-real-and-categorical-data/

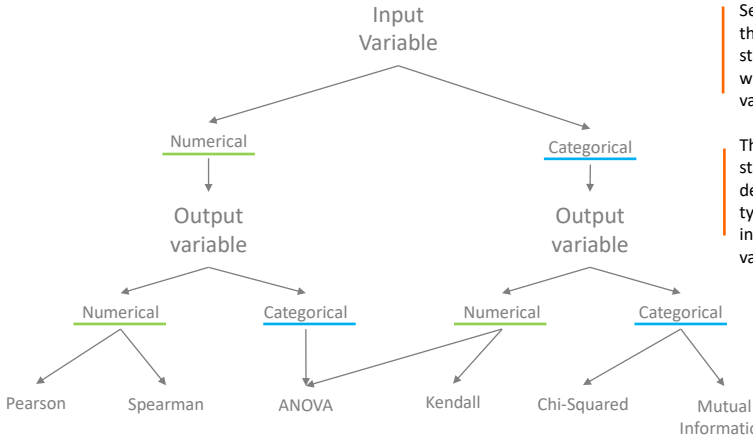
Feature Selection

- The Filter model
  - Chooses a subset of features, **without taking into consideration** which classifier is going to be used
    - It is focused exclusively in the **relationship between features and the target**
      - A feature can be regarded as irrelevant if it is **conditionally independent of the class labels**, i.e. the feature that has no influence on the class labels can be discarded.

Redundant	Excluded as a similar feature is already included
Irrelevant	Not predictive or excluded based on additional criteria

Feature Selection

- Filter models based on **statistical-based** feature selection methods involve:
  - Evaluating the relationship between each **input variable and the target variable**



Select input variables that have the strongest relationship with the target variable.

The choice of statistical methods depends on the data type of both the input and output variables

https://machinelearningmastery.com/feature-selection-with-real-and-categorical-data/

Feature Selection

- Feature Importance (filter model):
  - The relevance of a feature has to be measured based on evaluation criteria like:
    - Correlation
      - Linear relationship between feature and target
    - Dependence / Mutual information (MI) between features
      - If X and Y are independent then $MI = 0$
      - If they are dependent, one variable can provide information about the other (dependency) then $MI \geq 0$
  - Feature Selection should be applied to the entire training dataset.

Prepare Data for ML Algorithms

Training/Test Set
Missing Data
Encoding
Scaling
New Features
Imbalanced Datasets
Feature Selection
Transformation Pipelines

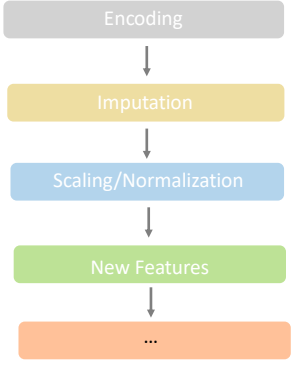
Transformation Pipeline

- In most machine learning projects the data that is unlikely to be in the ideal format for producing the best performing model.
  - The preparation of data for machine learning algorithms comprises of a set of transformation steps:
    - Encoding categorical variables,
    - Feature scaling
    - Normalization
    - New features/attributes
    - …



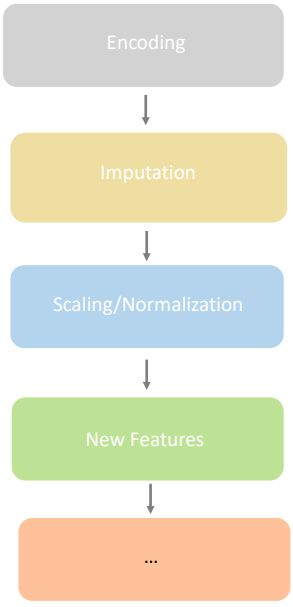
Transformation Pipeline

- These steps should **be executed in the right order.**
  - This sequence of transformations (missing data, encoding, scaling, …) is called **transformation pipeline**
    - It has to be applied **not only when training the model** but also when **new instances are applied to the model**.



Transformation Pipeline

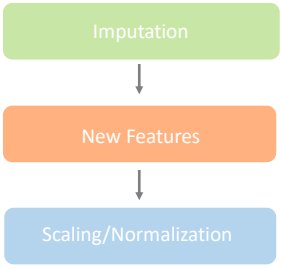
- The definition of pipelines has some advantages:
  - Workflow much easier to read and understand.
  - Implement a specific order of steps in the project.
  - Work much more reproducible
  - Scikit Learn provides the **Pipeline class** to help in the definition of transformation pipeline
    - https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html



Transformation pipeline example

Transformation Pipeline

- Pipeline
  - Sequence of operations





pipeline_num

pipeline specific for numeric variables

```
1  from sklearn.pipeline import Pipeline
2  from sklearn.impute import SimpleImputer
3  from sklearn.preprocessing import StandardScaler
4
5  pipeline_num=Pipeline([
6      ('imputer', SimpleImputer(strategy='median')),
7      ('attr_add',NewAttributes()),
8      ('std_scale',StandardScaler())
9  ])
10
11 all_attr_train_num=pipeline_num.fit_transform(train_num)
```

Transformation Pipeline

```python
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

pipeline_num=Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('attr_add',NewAttributes()),
    ('std_scale',StandardScaler())
])

all_attr_train_num=pipeline_num.fit_transform(train_num)
```

1 Imputation

2 New Features

3 Scaling/Normalization

Custom Transformer

```python
import numpy as np
from sklearn.base import BaseEstimator,TransformerMixin

rooms_ix=3
bedrooms_ix=4
population_ix=5
houses_ix=6

class NewAttributes(BaseEstimator,TransformerMixin):
    def __init__(self, bedrooms=True):
        self.a=bedrooms

    def fit(self,X, y=None):
        return self

    def transform(self,X, y=None):
        rooms_house=X[:,rooms_ix]/X[:,houses_ix]
        people_house=X[:,population_ix]/X[:,houses_ix]
        bedrooms_room=X[:,bedrooms_ix]/X[:,rooms_ix]

        return np.c_[X,rooms_house,people_house,bedrooms_room]
```


Transformation Pipeline

- ***ColumnTransformer***
  - It allows the implementation of a single transformer able to handle all columns applying the appropriate transformations to each column/feature according to its nature

Numeric

Categorical

- The *ColumnTransformer* pipeline can contain other pipelines
  - It applies each transformer to the appropriate columns and concatenates the outputs
    - https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

Transformation Pipeline

- ***ColumnTransformer***
  - Data preprocessing according to the column type.

```
1  from sklearn.compose import ColumnTransformer
2  from sklearn.preprocessing import OneHotEncoder
3
4  attr_num=train_num.columns
5  attr_cat=['ocean_proximity']
6
7  global_pipeline=ColumnTransformer([
8      ('num',pipeline_num,attr_num),
9      ('cat',OneHotEncoder(),attr_cat)
10 ])
11 train_preprocessed=global_pipeline.fit_transform(train_instances)
```

Numeric Columns

Categorical Columns

- https://scikit-learn.org/stable/auto_examples/compose/plot_column_transformer_mixed_types.html

Training and Evaluating a Model

Training and Evaluating a Model

- After the preparation of data, the following step is training and evaluating a model
  - Regression Problem
    - Linear Regression Model (first try! Iterative process)

```
1 from sklearn.linear_model import LinearRegression
```

Trained linear regression model!

- Train and evaluate on **the training set**

```
1 from sklearn.metrics import mean_squared_error
```

- Root Mean Squared Error (RMSE): **67593.21**

Poor result!
An average error of approximately $67600 is not acceptable regarding that the mean value of an house is $206855
It represents 32% of the mean.

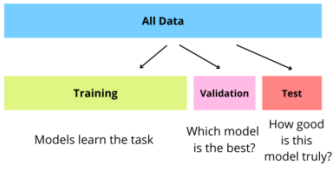
Training and Evaluating a Model

- The previous result suggest a typical situation of underfitting that can be originated by:
  - Features do not provide enough information to make good predictions
  - The model is not powerful enough to explore the data
    - Analyzing the same problem but with a Decision Tree Regressor:

```
1 from sklearn.tree import DecisionTreeRegressor
```

- Train and evaluate on **the training set**
  - Root Mean Squared Error (RMSE): **0** ?

- Conclusions?
  - Probably overfitting situation.
    - How to confirm? Validation set!
  - Test set is out of this process.

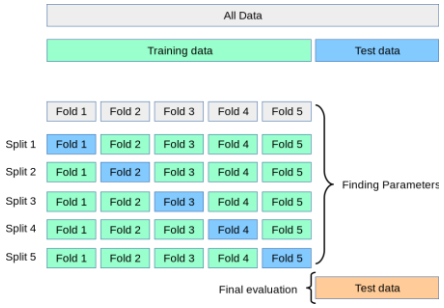


https://galaxyinferno.com/what-is-validation-data-used-for-machine-learning-basics/

Training and Evaluating a Model

- To create the validation (dev) set, a new split to the training set into a smaller training set and a validation set could be applied
  - The goal is to train the model with the new training set and evaluate it against the new validation set.
  - As an alternative approach to better evaluate the model, a **k-fold cross validation** can be implemented

Cross-validation scoring may be used to compare the efficacy of **various hyperparameter settings**



https://scikit-learn.org/stable/modules/cross_validation.html

Training and Evaluating a Model

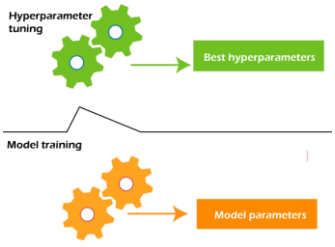
- Results of **Decision Tree (DT)** Regression model based on Cross Validation
  - Root Mean Squared Error (RMSE): **68509.71**
    - It was as overfitting situation.
    - The performance is clearly worse than before and it seems even worse than linear regression
- Results of **Linear Regression** model based on Cross Validation
  - Root Mean Squared Error (RMSE): **69042.7**
- Results of a more complex model, **Random Forest Regressor**, model based on Cross Validation
  - Root Mean Squared Error (RMSE): **50038.81**
    - A more complex algorithm produces better results, but even in this situation the results can be significantly improved.

Training and Evaluating a Model

- The goal is to achieve a short list of promising models:
  - Decision Tree Regressor
  - Linear Regression Model
  - Random Forest Regressor

Models' short-list

that might be refined through the tuning of hyperparameters



https://www.javatpoint.com/hyperparameters-in-machine-learning

Select and Train a Model

Fine Tuning of a Model

Fine Tuning of a Model

- After the selection of a model or a short-list of models, fine tuning is often required
  - Hyperparameters of the model have to be adjusted
    - Manual Tuning
      - Time consuming and tedious work
    - Grid Search
      - Automatic evaluation of all the possible combinations of hyperparameters defined by the user

```
1 from sklearn.model_selection import GridSearchCV
```


Fine Tuning of a Model

- Grid Search
  - Example: Hyperparameter tuning for Random Forest

```
1 from sklearn.model_selection import GridSearchCV
2
3 param_grid=[{'n_estimators':[3,10,30],'max_features':[2,4,6,8]},
4             {'bootstrap':[False],'n_estimators':[3,10], 'max_features': [2,3,4]}]
```

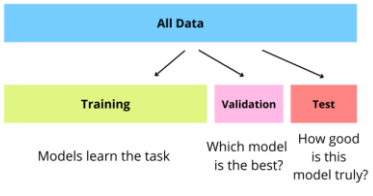
    - The grid search explores 12 combinations (first dictionary) plus 6 combinations defined in the second one (with bootstrap resampling disabled)
      - These 18 different combinations are applied to a RandomForestRegressor
    - If each model is trained 5 times (five fold cross validation), there are $18 \times 5 = 90$ rounds of training
      - Time consuming but very important process!!

Select and Train a Model

Fine-Tune a Model
Evaluation on the Test Set

Evaluation of the Test Set

- Final Performance Evaluation
  - After select/tune the model, the evaluation against the test set follows the steps:
    - Prepare the test set



    - transformations through the pipeline
      - the previous transformation pipeline is also applied to the test set

```
1 test_preprocessed=global_pipeline.fit_transform(test_instances)
```

Unseen data applied to the transformation pipeline (previously implemented)

Evaluation of the Test Set

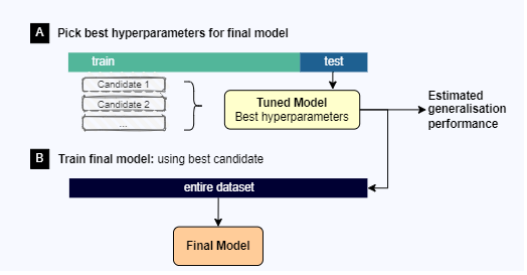
- Apply the model to the prepared test set
- Calculate RMSE (y_test contains the real labels)
  - Root Mean Squared Error (RMSE): **49228.65**
    - This is an example of a less frequent situation, where the error in the test set is slightly lower than in the validation set.

Deployment and Monitor

Deployment and Monitor

- The final model is obtained applying the chosen machine learning procedure on the **entire dataset**.

*"Your model will likely perform better **when trained on all of the available data** than just the subset used to estimate the performance of the model. "*



https://docs.graphext.com/guides/model_train_eval/#model-tuning-and-evaluation

- Final machine learning model preparation:
  - https://machinelearningmastery.com/train-final-machine-learning-model/

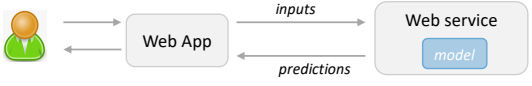
Deployment and Monitor

- Deployment
  - To save the SciKit-Learn model as well as all the preprocessing and transformation pipeline

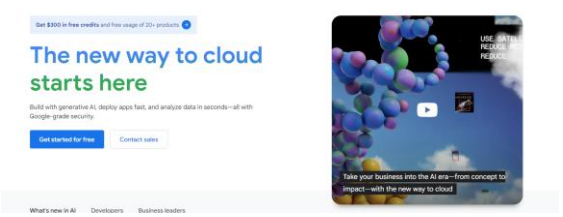
https://scikit-learn.org/stable/modules/model_persistence.html

https://machinelearningmastery.com/save-load-machine-learning-models-python-scikit-learn/

  - To wrap the model in a dedicated web service

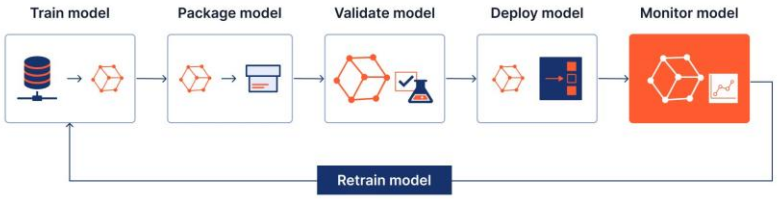


  - To deploy the model on the Cloud
    - https://cloud.google.com/



Deployment and Monitor

- Monitor
  - machine learning models is related with the monitoring of the quality of decision making that the model achieves.

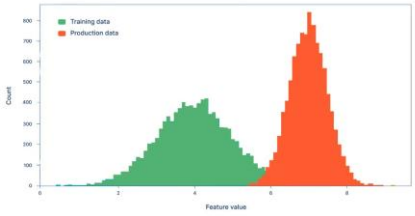


https://dysnix.com/blog/ml-model-monitoring-in-production

https://christophergs.com/machine%20learning/2020/03/14/how-to-monitor-machine-learning-models/

Deployment and Monitor

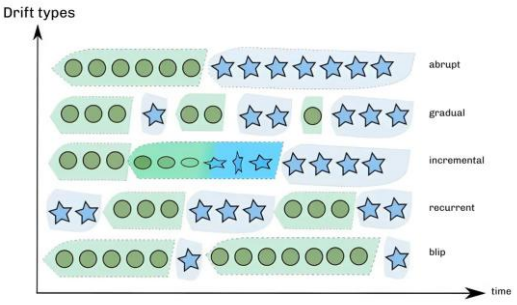
- The quality of predictions is a function of:
  - The **quality of the data fed** to models, as that data might come from different sources (user, other systems, sensors, …).
    - Monitoring an ML system also involves monitoring all the data sources that feed the model.
  - **Modeling assumptions**
    - Since the real world is dynamic, the **learned relationships** between inputs and outputs from historical data **might have to be updated**.
    - Model **performance naturally degrades with time**, so the model must be adapted to changing conditions.



https://dysnix.com/blog/ml-model-monitoring-in-production

Deployment and Monitor

- Modeling assumptions



"For example, a recommendation system for e-commerce might perform well during the holiday season but falter as customer preferences shift afterward. Monitoring allows teams to detect these changes early, enabling retraining or fine-tuning to keep the model relevant and effective."

https://dysnix.com/blog/ml-model-monitoring-in-production

Deployment and Monitor

- A robust and continuous monitoring is a key issue in the detection of these variations/changing conditions.
  - Changes in any part of the system (hyper-parameters, learning settings, sampling methods, and data selection) can cause unpredictable changes to model output.

CACE: Changing Anything Changes Everything

